

Neural_network

August 12, 2026

0.1 Handwritten Digit Recognition using Neural Networks

```
[1]: #imports
import pandas as pd
import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Input
from sklearn.model_selection import train_test_split
```

load the MNIST dataset using sklearn

```
[2]: data = mnist.load_data()
X = data[0][0]
Y = data[0][1]

X.shape, Y.shape
```

Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz>
11490434/11490434 12s
1us/step

```
[2]: ((60000, 28, 28), (60000,))
```

split the dataset into training and testing sets

```
[3]: (x_train, x_test, y_train, y_test) = train_test_split(X, Y, test_size=0.2,
↳ random_state=42)
x_train.shape, x_test.shape, y_train.shape, y_test.shape
```

```
[3]: ((48000, 28, 28), (12000, 28, 28), (48000,), (12000,))
```

preprocess the data by normalizing the pixel values to be between 0 and 1

```
[4]: x_train = x_train.astype('float32') / 255.0
x_test = x_test.astype('float32') / 255.0

#flatten the images to 1D arrays of size 784 (28x28)
x_train = x_train.reshape(-1, 28*28)
```

```
x_test = x_test.reshape(-1, 28*28)

#convert the labels to one-hot encoded vectors
y_train = pd.get_dummies(y_train).values
y_test = pd.get_dummies(y_test).values
y_train.shape, y_test.shape
```

[4]: ((48000, 10), (12000, 10))

train the model using a simple feedforward neural network with one hidden layer

```
[5]: model = Sequential()
model.add(Input(shape=(784,)))
model.add(Dense(128, activation='relu'))
model.add(Dense(10, activation='softmax'))
model.compile(optimizer='adam', loss='categorical_crossentropy',
              metrics=['accuracy'])
model.fit(x_train, y_train, epochs=10, batch_size=32, validation_split=0.2)
```

C:\Users\chalo\anaconda3\envs\machine-learning\Lib\site-packages\keras\src\layers\core\dense.py:107: UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.

```
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.11. Even if CUDA/cuDNN are installed, GPU will not be used. Please use WSL2 or the TensorFlow-DirectML plugin.

Epoch 1/10

```
1200/1200          9s 6ms/step -
accuracy: 0.9097 - loss: 0.3147 - val_accuracy: 0.9465 - val_loss: 0.1888
```

Epoch 2/10

```
1200/1200          7s 6ms/step -
accuracy: 0.9578 - loss: 0.1461 - val_accuracy: 0.9570 - val_loss: 0.1416
```

Epoch 3/10

```
1200/1200          7s 6ms/step -
accuracy: 0.9706 - loss: 0.1006 - val_accuracy: 0.9643 - val_loss: 0.1144
```

Epoch 4/10

```
1200/1200          7s 6ms/step -
accuracy: 0.9775 - loss: 0.0753 - val_accuracy: 0.9658 - val_loss: 0.1122
```

Epoch 5/10

```
1200/1200          7s 5ms/step -
accuracy: 0.9832 - loss: 0.0580 - val_accuracy: 0.9700 - val_loss: 0.1027
```

Epoch 6/10

```
1200/1200          7s 6ms/step -
accuracy: 0.9870 - loss: 0.0436 - val_accuracy: 0.9710 - val_loss: 0.0929
```

Epoch 7/10

```
1200/1200          7s 6ms/step -
accuracy: 0.9901 - loss: 0.0352 - val_accuracy: 0.9725 - val_loss: 0.0929
```

```
Epoch 8/10
1200/1200          8s 7ms/step -
accuracy: 0.9924 - loss: 0.0270 - val_accuracy: 0.9736 - val_loss: 0.0924
Epoch 9/10
1200/1200          9s 5ms/step -
accuracy: 0.9937 - loss: 0.0217 - val_accuracy: 0.9725 - val_loss: 0.0973
Epoch 10/10
1200/1200          7s 6ms/step -
accuracy: 0.9950 - loss: 0.0188 - val_accuracy: 0.9708 - val_loss: 0.1018
```

```
[5]: <keras.src.callbacks.history.History at 0x188923630e0>
```

evaluate the model on the test set

```
[6]: test_loss, test_acc = model.evaluate(x_test, y_test)
      print('Test accuracy:', test_acc)
      print('Test loss:', test_loss)
```

```
375/375           1s 4ms/step -
accuracy: 0.9738 - loss: 0.0986
Test accuracy: 0.9738333225250244
Test loss: 0.09863821417093277
```